Solution: Sort Characters By Frequency (easy)

Problem Statement

Given a string, arrange its characters in descending order based on the frequency of each character. If two characters have the same frequency, their relative order in the output string can be arbitrary.

Example

1. **Input**: "apple"
   - **Expected Output**: "ppale" or "ppela"
   - **Justification**: The character 'p' appears twice, while 'a', 'l', and 'e' each appear once. Thus, 'p' should appear before the other characters in the output.
2. **Input**: "banana"
   - **Expected Output**: "aaannb"
   - **Justification**: The character 'a' appears three times, 'n' twice and 'b' once.
3. **Input**: "aabb"
   - **Expected Output**: "aabb" or "bbaa"
   - **Justification**: Both 'a' and 'b' appear twice, so they can appear in any order in the output.

Solution

The solution involves three key steps. First, count the frequency of each character in the string. This can be done using a hashmap where keys are characters and values are their respective counts. Second, build a max heap (priority queue) where each element is a character, and the heap is sorted based on the frequency of characters. The character with the highest frequency will be at the top. Lastly, construct the result string by repeatedly removing the top element of the heap (the most frequent character) and appending it to the result string until the heap is empty. This process ensures that characters are added to the result string in descending order of their frequencies.

Detailed Breakdown:

1. **Building a Frequency Map**: Begin by iterating over the characters in the string. As you traverse, construct a frequency map (or dictionary) that associates each character with its occurrence count.
2. **Constructing a Max Heap**: Next, utilize the frequency map to build a max heap. Each item in this heap will be a character-frequency pair, but the heap will be organized based on the frequencies. This ensures that characters with higher frequencies will reside at the top of the heap.
3. **Building the Result String**: Start an iterative process where you repeatedly pop the heap to get the character with the highest current frequency. Each time you pop, append the character to the result string as many times as its frequency indicates. This ensures that characters with higher frequencies are placed before those with lower frequencies in the result.
4. **Completion**: Once the heap is exhausted, your result string is complete, containing characters sorted by their frequencies in descending order. This string is then returned as the output.

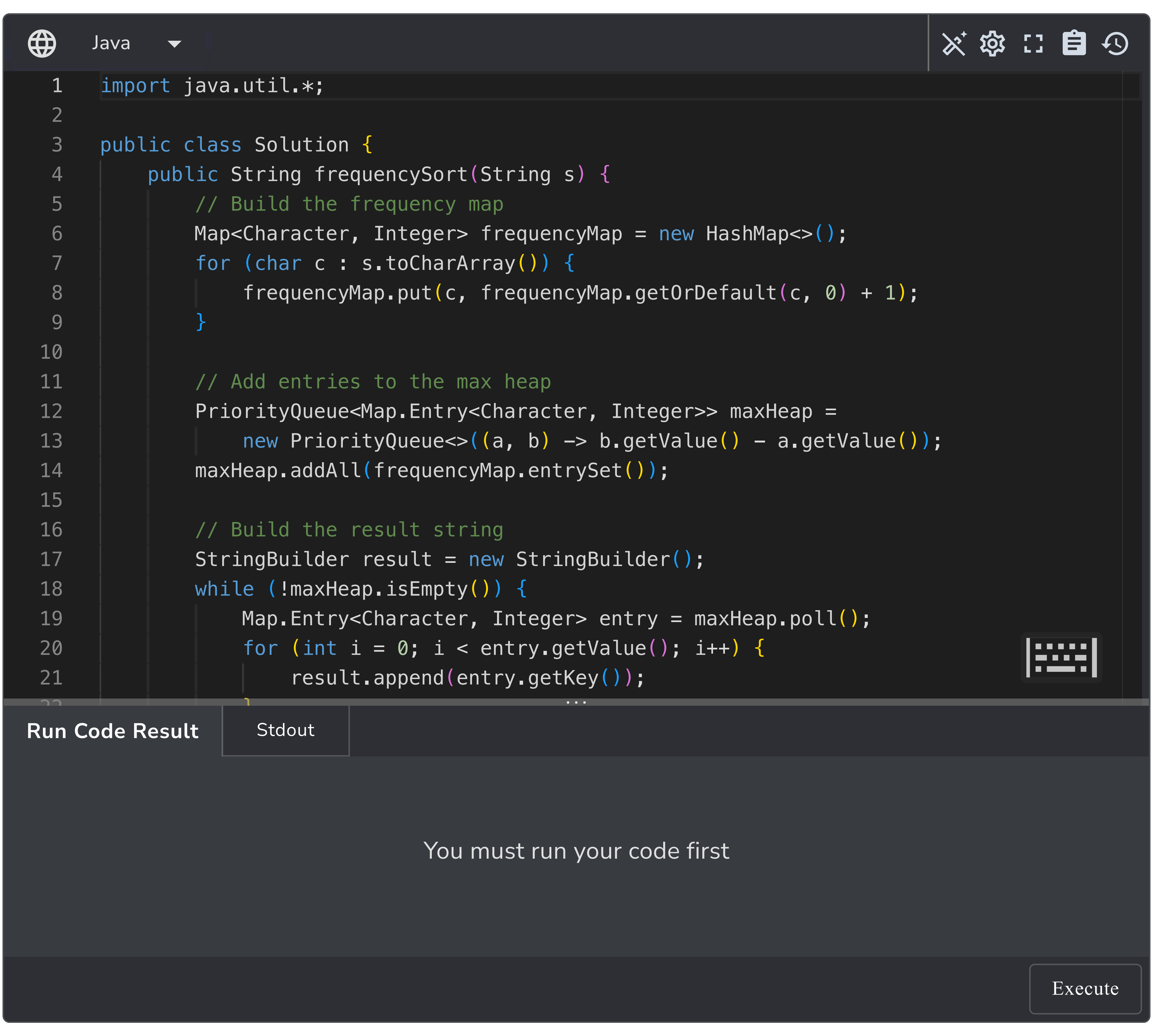
The utilization of a max heap is crucial here, as it allows efficient retrieval of the character with the highest frequency at any given point in the process, ensuring the desired order in the result string.

Algorithm Walkthrough:
Using the input "apple":

- Construct a frequency map: {'a':1, 'p':2, 'l':1, 'e':1}
- Push the characters onto a max heap: [(2, 'p'), (1, 'a'), (1, 'l'), (1, 'e')]
- Pop the heap and build the result string:
  - Pop: (2, 'p') -> result = "pp"
  - Pop: (1, 'a') -> result = "ppa"
  - Pop: (1, 'l') -> result = "ppal"
  - Pop: (1, 'e') -> result = "ppale"

Code

Here is the code for this algorithm:



```java
import java.util.*;

public class Solution {
    public String frequencySort(String s) {
        // Build the frequency map
        Map<Character, Integer> frequencyMap = new HashMap<>();
        for (char c : s.toCharArray()) {
            frequencyMap.put(c, frequencyMap.getOrDefault(c, 0) + 1);
        }

        // Add entries to the max heap
        PriorityQueue<Map.Entry<Character, Integer>> maxHeap =
            new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());
        maxHeap.addAll(frequencyMap.entrySet());

        // Build the result string
        StringBuilder result = new StringBuilder();
        while (!maxHeap.isEmpty()) {
            Map.Entry<Character, Integer> entry = maxHeap.poll();
            for (int i = 0; i < entry.getValue(); i++) {
                result.append(entry.getKey());
```

You must run your code first

Time Complexity:

1. **Counting the Frequency of Characters:** We traverse the string of length (n) once to count the frequency of each character. This process has a time complexity of ($O(n)$).
2. **Inserting into a Heap/Min-Priority Queue:** For every unique character in the input string, we insert it into a priority queue based on its frequency. The insertion of a single element into a priority queue takes ($O(\log m)$) time where (m) is the number of unique characters in the string. In the worst case, (m) can be the size of the character set, but it's much smaller than (n) in most practical scenarios. The overall time for this step is ($O(m \log m)$).
3. **Building the Result String:** While the heap is not empty, we pop the most frequent character and append it to our result string. Popping from a heap takes ($O(\log m)$) time, and we do it (m) times. Appending characters to the result string can be done in ($O(1)$) time. Therefore, this step overall is ($O(m \log m)$).

Combining all steps, the dominant factor for time complexity is the process of pushing and popping from the priority queue. Therefore, the overall time complexity is ($O(n + m \log m)$). Since (m) is often much smaller than (n) (especially for large strings), the time complexity can be approximated as ($O(n)$) in many cases, but it's more accurate to represent it as ($O(n + m \log m)$).

Space Complexity:

1. **Storing Frequencies:** We use a hashmap or an equivalent data structure to store the frequency of each character. In the worst case, this will take up space proportional to the number of unique characters, (m). This gives us a space complexity of ($O(m)$).
2. **Priority Queue:** We also use a priority queue to store unique characters based on their frequency. In the worst case, this queue can have all unique characters, adding another ($O(m)$) space.
3. **Result String:** We create a new string to store our result. In the worst case, this string will be of length (n), the length of the original string, giving us a space complexity of ($O(n)$).

Adding up all the components, the total space complexity is ($O(n + 2m)$). However, for the sake of big O notation, this simplifies to ($O(n + m)$).

To summarize:

- **Time Complexity:** ($O(n + m \log m)$)
- **Space Complexity:** ($O(n + m)$)

← Previous
Next →
Sort Characters By Frequency (Easy)
Minimum Cost To Connect Sticks(Medium)
☐ Mark as Completed